



WEEK THREE

從系統骨架到可交付產品

在第三週，我們會把前兩週做出的系統，真正推到「可以交付、可以展示、可以被評估」的狀態。這一週，你不再只是把功能補齊，而是第一次完整走完一條產品路徑：從需求、結構、前後端分工、部署、展示，到最後的商業判斷。

在這一週，我們會完成三種不同型態的 Web App 實作：

- 一個主線 MVP，讓你真正做完一個可部署、可展示的產品
- 一個世界同步型系統，理解為什麼後端不是選項，而是必要
- 一個決策型系統，學會哪些商業邏輯不能寫在前端

這些 App 的目的，不是數量，而是讓你開始分得清什麼該做成產品，什麼該停在作品。

別忘了新增本週實作資料夾：`vibe-coding-week3`

CHAPTER 0 從練習者到交付者：WEEK 3 的角色轉換	8
0.1 先說清楚：Week 3 跟前兩週「哪裡不一樣」	8
0.2 Demo、作品、MVP，其實是三種完全不同的東西	9
0.3 從這一刻開始，你寫的不是「自己的東西」	10
0.4 Vibe Coding 在 Week 3 的定位會被重新定義	11
0.5 今天你會看到「很多 Demo」，但目的只有一個	11
0.6 帶著這三個問題，進入 Week 3	12
CHAPTER 1 從 DEMO 到 MVP：什麼時候該認真了	13
1.1 Demo 為什麼不夠	13
1.2 什麼叫 MVP (Minimum Viable Product)	13
1.3 Demo、作品、MVP 的差別	14
1.4 什麼時候 Demo 應該升級成 MVP	14
1.5 為什麼一升級成 MVP，就會碰到後端	15
1.6 MVP 不是功能變多，而是責任變清楚	15
1.7 本週你會看到的 MVP 與 Demo，該怎麼看	16
1.8 帶著這個判斷，進入下一章	16
CHAPTER 2 主線 MVP 拆解：節奏型行為追蹤系統	17
2.1 為什麼我們需要一個「主線 MVP」	17
2.2 MVP 的核心定位：我們在追蹤的不是行為，而是「節奏」	17
2.3 MVP 解決的是哪一個真實問題	18
2.4 MVP 的最小使用流程	18
2.5 為什麼這個 MVP 一定需要後端	19
2.6 前端在這個 MVP 中的角色	19

2.7 後端在這個 MVP 中的角色	19
2.8 MVP 的「狀態」設計，比功能更重要	20
2.9 這個 MVP 如何延伸成不同產品	20
2.10 為什麼這個 MVP 適合當 Week 3 主線	21
2.11 這個 MVP 的最低完成標準	21
2.12 帶著這個 MVP，進入下一章	22
CHAPTER 3 前端與後端：不是技術選擇，是責任切分	23
3.1 為什麼前端與後端一定要分清楚	23
3.2 一個簡單的判斷原則	23
3.3 前端在 MVP 中該負責的事情	23
3.4 前端不該承擔的事情	24
3.5 後端在 MVP 中存在的真正理由	24
3.6 用主線 MVP 看前後端分工	25
3.7 為什麼「一致性」一定是後端責任	25
3.8 常見錯誤：把後端邏輯藏在前端	26
3.9 在本課中，哪些 Web App 一定需要後端	26
3.10 這一章你應該能做到的判斷	26
3.11 帶著這個分工，進入下一章	27
CHAPTER 4 BYOK 是產品設計問題	28
4.1 什麼是 BYOK (Bring Your Own Key)	28
4.2 為什麼在 Demo 階段很容易忽略 BYOK	28
4.3 如果不想 BYOK，會發生什麼事	29
4.4 BYOK 本質上是在做一個責任切分	29

4.5 常見的三種 BYOK 設計方式	30
4.5.1 完全前端 BYOK	30
4.5.2 後端轉接型 BYOK (最適合 MVP)	30
4.5.3 平台代管型 (不適合課堂 MVP)	31
4.6 為什麼 BYOK 一定要搭配後端設計	32
4.7 在本課的 Web App 中，BYOK 會怎麼出現	32
4.8 BYOK 沒想清楚，商業評估就一定失真	33
4.9 本章你應該能做出的判斷	33
4.10 帶著 BYOK 的設計意識，進入實作	33
CHAPTER 5 WEB APP 實作：用 Q 做出可部署的 MVP	34
5.1 本章你在做的不是練習，是一個 MVP	34
5.2 本章實作方法 (請照順序)	34
5.3 本章你可以選擇的部署方式 (先講清楚)	35
Web App #1 主線 MVP：節奏型行為追蹤系統 (必做)	36
5.4 這個 Web App 的最低完成標準	36
5.5 Step 1：用 Q 建立專案骨架	36
5.6 Step 2：用 Q 定義 API 契約 (前後端合體核心)	37
5.7 Step 3：用 Q 產出後端 (先能跑)	38
5.8 Step 4：用 Q 產出前端 (只做輸入與顯示)	39
5.9 Step 5：用 Q 完成前後端合體與測試	40
5.10 Step 6：用 Q 部署後端 (Cloud Run 或 Firebase)	41
5.11 Step 7：部署前端並完成對外展示	42
5.12 主線 MVP 完成標準 (請自我檢核)	42
Web App #2、#3 (延伸 Demo)	42

5.13 本章你真正學會的是什麼	43
Web App #2 即時匿名回饋牆 (多人同步型)	44
#2.1 這個 Web App 在做什麼	44
#2.2 最低完成標準 (做到這裡就算過關)	44
#2.3 Step A 用 Q 建立專案骨架	45
#2.4 Step B 用 Q 定 API 契約	46
#2.5 Step C 用 Q 產出後端 (資料一致性)	47
#2.6 Step D 用 Q 產出前端 (只顯示)	48
#2.7 Step E 用 Q 合體與測試	48
#2.8 Step F 部署 (選一種)	49
Web App #3 決策型 Web App (後端判斷)	50
#3.1 這個 Web App 在做什麼	50
#3.2 最低完成標準	50
#3.3 Step A 用 Q 建立專案骨架	50
#3.4 Step B 用 Q 定 API 契約	51
#3.5 Step C 用 Q 產出後端 (規則集中)	51
#3.6 Step D 用 Q 產出前端 (只顯示結果)	52
#3.7 Step E 合體與驗證	52
#3.8 Step F 部署 (推薦 Cloud Run)	52
CHAPTER 6 GIT 同步與交付：讓你的 WEB APP 不是只活在你電腦	53
6.1 為什麼「能跑」不等於「完成」	53
6.2 本章你要做到的三件事	53
6.3 Step 1 用 Q 幫你建立「可交付」的 Git 專案結構	54
6.4 Step 2 用 Q 幫你寫 .gitignore (避免災難)	55
6.5 Step 3 用 Q 幫你寫「真的有人看得懂」的 README	56
6.6 Step 4 用 Q 建立「安全的環境變數策略」	57
6.7 Step 5 用 Q 規劃「不會後悔的 commit 節奏」	58
6.8 Step 6 用 Q 模擬「交付給別人」的檢查流程	58
6.9 本章完成標準 (請誠實對自己)	59
6.10 為什麼這一章比寫程式還重要	59

CHAPTER 7 部署與展示：把你的 WEB APP 變成「能給人看的東西」	60
7.1 為什麼「能部署」不等於「能展示」	60
7.2 本章你要完成的三個展示任務	60
7.3 展示前你一定要先做的「收斂」	61
7.4 Step 1 用 Q 幫你定「展示版本的目標」	61
7.5 Step 2 用 Q 幫你調整「展示專用 UI」	62
7.6 Step 3 用 Q 幫你準備「展示說詞」	63
7.7 Step 4 用 Q 設計「展示用測試資料」	63
7.8 Step 5 用 Q 做「展示前最後檢查」	64
7.9 不同 Web App 的展示重點提醒	64
7.10 本章完成標準	65
7.11 為什麼展示是產品能力的一部分	65
CHAPTER 8 VIBE CODING 常見翻車點與修正方式	66
8.1 為什麼「會用 Q」反而更容易翻車	66
8.2 翻車類型一：指令太大，一次要太多	67
8.3 翻車類型二：沒有 API 契約就開始寫	68
8.4 翻車類型三：前端開始偷做後端的事	69
8.5 翻車類型四：改一點東西，整個系統崩	70
8.6 翻車類型五：錯了，但不知道錯在哪一層	71
8.7 翻車類型六：Q 寫得越來越不像你	72
8.8 本章的核心不是避免錯誤，而是避免失控	73
CHAPTER 9 商業評估：AI CODING 產品要怎麼活下來	74
9.1 為什麼 AI Coding 產品「做得出來」通常不是問題	74

9.2 AI Coding 產品最常見的三個錯誤期待	74
9.3 AI Coding 產品的五種常見商業模式 (你一定要認得)	75
9.3.1 工具型 (Tool SaaS)	75
9.3.2 BYOK 平台型 (AI 功能層)	75
9.3.3 專案型 / 顧問型 (Solution)	76
9.3.4 內容 × 工具混合型	77
9.3.5 展示型 (不賣產品)	77
9.4 用三個問題，快速篩選商業可能性	78
9.5 AI 成本結構，你一定要算的不是營收	78
9.6 用 Q 幫你選「最適合現在的商業定位」	79
9.7 本課真正想讓你帶走的商業觀念	79
9.8 結課總結	79
三種 AI 產品商業路線圖 (實作後的真實選項)	80
路線一 BYOK 工具型 (最適合課堂 MVP)	80
路線二 專案型 / 顧問型 (最快變現)	82
路線三 內容 × 工具混合型 (個人品牌最適合)	84

Chapter 0 從練習者到交付者：Week 3 的角色轉換

在進入部署、後端、商業與大量 Demo 之前，我們需要先確認一件事：

你現在，用的是什麼身分在寫程式？

0.1 先說清楚：Week 3 跟前兩週「哪裡不一樣」

在 Week 1，你關心的是：

- 我能不能照著做出來？
- 這段 code 為什麼會動？
- AI 可不可以幫我補齊？

在 Week 2，你開始關心的是：

- 系統會不會亂掉？
- 狀態、流程、畫面能不能對得起來？
- 我是不是在「堆功能」？

而從 Week 3 開始，你要關心的是：如果這個東西交到別人手上，我能不能為它現在的樣子負責？

這一週，我們不再用「作業完成度」來看成果，而是用「是否具備交付條件」來判斷。

0.2 Demo、作品、MVP，其實是三種完全不同的東西

在開始今天的內容前，請先分清楚這三個詞。

Demo 是什麼？

- 證明「做得到」
- 可以不穩
- 可以只在你電腦上

作品是什麼？

- 有完成感
- 可以展示
- 但不一定能被使用

MVP 是什麼？

- 有清楚的使用情境
- 有你能負責的範圍
- 壞了你知道為什麼
- 有人「真的用得上」

Week 3 的所有內容，都是圍繞在「怎麼把 Demo 推到 MVP 的最低門檻」。

0.3 從這一刻開始，你寫的不是「自己的東西」

請你先接受一個現實：

只要一個系統

- 要給別人用
- 要放到網路上
- 要接 API、資料庫或金鑰

那它就不再只是你的練習。

這也是為什麼，從 Week 3 開始，我們會正式談到：

- 後端存在的必要性
- BYOK 與成本責任
- Git 與同步
- 商業評估與停損判斷

這些東西**不是進階技巧**，而是「你開始接近真實世界時，一定會撞到的牆」。

0.4 Vibe Coding 在 Week 3 的定位會被重新定義

在前兩週，你可能已經體會到 Vibe Coding 的爽感：

- 改得快
- 試得多
- 靈感來就寫

但從這一週開始，我們要補上一句非常重要的限制：

Vibe Coding 可以用來探索，但不能用來逃避責任

所以接下來你會看到的 Demo，會刻意示範兩件事：

- 什麼時候 Vibe Coding 非常好用
- 什麼時候 Vibe Coding 會直接把產品帶進死路

0.5 今天你會看到「很多 Demo」，但目的只有一個

今天的課程會 demo：

- 一個主線 MVP 產品
- 多個完全不同物種的 Web App
- 多種「非用後端不可」的情境

請你記住一件事：

這些 Demo 不是要你全部學會，而是要你學會**判斷**。判斷：

- 什麼時候該加後端
- 什麼時候該停
- 什麼時候不該再「只是玩」

0.6 帶著這三個問題，進入 Week 3

在進入 Chapter 1 之前，請你先把這三題放在心裡：

1. 這個東西如果壞了，誰會受影響？
2. 如果我要繼續做，哪些地方我必須負責？
3. 哪一個決定，會決定它能不能活下來？

接下來的每一個章節、每一個 Demo，都會不斷回到這三個問題。

Week 3 不只讓你寫更多程式，還會讓你開始知道什麼樣的程式，不該隨便寫

Chapter 1 從 Demo 到 MVP：什麼時候該認真了

1.1 Demo 為什麼不夠

在前兩週，你已經做出「會動的東西」。它們可能可以展示、可以操作，甚至看起來很完整。

但 Demo 有一個很大的特徵：

- 它只需要對「當下」負責
- 它只需要在「你的電腦」上成立
- 它不需要考慮壞掉之後怎麼辦

Demo 的目的只有一個：證明「做得到」。而這個目的，在 Week 2 就已經完成了。

1.2 什麼叫 MVP (Minimum Viable Product)

MVP 不是「半成品」，也不是「簡單版」。

MVP 的意思是：這是一個已經具備最低可存活條件的產品。

一個 MVP 至少要回答三件事：

1. 誰會用它？
2. 用它要解決什麼問題？
3. 壞了，你知不知道為什麼？

只要有一題答不出來，它就還停留在 Demo 階段。

1.3 Demo、作品、MVP 的差別

請用下面這個方式幫自己做判斷：

- Demo
 - 重點：能不能做出來
 - 可以不穩定
 - 可以只存在你電腦
- 作品
 - 重點：能不能展示
 - 有完成感
 - 但不一定能被真實使用
- MVP
 - 重點：能不能被使用
 - 有清楚邊界
 - 有你願意負責的範圍

Week 3 的目標不是把每個 Demo 都變成 MVP，
而是學會判斷：哪一個值得被推進，哪一個該停在 Demo。

1.4 什麼時候 Demo 應該升級成 MVP

如果你的系統開始出現下面這些情況，就代表它正在跨過那條線：

- 不只你一個人會用
- 資料需要被保存，而不是重整就消失
- 使用結果會影響下一步行動
- 你開始在意「它如果壞了怎麼辦」

這些都不是技術問題，而是產品已經開始進入真實世界的訊號。

1.5 為什麼一升級成 MVP，就會碰到後端

在 Demo 階段，很多事情可以先不用管：

- 資料存在記憶體
- 邏輯寫在前端
- 每次重來都沒差

但只要你開始做 MVP，就會立刻遇到這些問題：

- 資料要不要留？
- 不同人看到的東西要不要一致？
- 判斷邏輯該放在哪？
- 成本跟風險誰負責？

這些問題，前端不該也無法獨立承擔。

這也是為什麼，Week 3 會正式進入後端與系統責任的討論。

1.6 MVP 不是功能變多，而是責任變清楚

一個常見的誤解是：「我要再加幾個功能，才算 MVP。」

事實正好相反，MVP 的關鍵不是功能數量，而是：

- 哪些事你願意負責
- 哪些事你現在明確不做
- 哪些風險你已經意識到

一個功能很多、但責任模糊的系統，比一個功能很少、但邊界清楚的系統，更不像產品。

1.7 本週你會看到的 MVP 與 Demo，該怎麼看

接下來的課程中，你會看到：

- 一個主線 MVP 產品
- 多個完全不同類型的 Web App Demo
- 多種「非用後端不可」的情境

請記住一件事：這些示範不是要你全部照做，而是要你練習判斷：

- 這個東西，值不值得變成 MVP？
- 如果要變，責任會落在哪？
- 如果不變，停在 Demo 是否反而是好選擇？

1.8 帶著這個判斷，進入下一章

在進入 Chapter 2 之前，請先問自己：

- 我目前最接近 MVP 的作品是哪一個？
- 如果它今天交到別人手上，我最不安心的是哪一段？
- 那一段，是前端問題，還是後端責任？

接下來的章節，會一一幫你拆解這些問題。

Chapter 2 主線 MVP 拆解：節奏型行為追蹤系統

2.1 為什麼我們需要一個「主線 MVP」

在 Week 3，你會看到很多不同的 Demo、不同類型的 Web App。如果沒有一個共同參考點，很容易只剩下「看熱鬧」。

因此，我們需要一個**主線 MVP**，用來回答三個問題：

- 什麼樣的系統，值得從 Demo 推進成產品？
- 什麼時候，前端已經不夠？
- 後端在產品中，到底負責什麼？

這個主線 MVP，不是為了功能完整，而是為了**結構完整**。

2.2 MVP 的核心定位：我們在追蹤的不是行為，而是「節奏」

這個 MVP 的目的，不是單純記錄你做了什麼。它關心的是：

- 行為是否穩定
- 節奏是否斷裂
- 這樣的狀態，能不能長期維持

換句話說，它追蹤的不是「一次結果」，而是一段時間內的變化。這也是為什麼，這個系統一開始就不適合只做成前端 Demo。

2.3 MVP 解決的是哪一個真實問題

請先聚焦一個非常單純的問題：如果一個人（或一個系統）表面上看起來很努力，但節奏其實一直在崩，我們有沒有辦法提早看出來？

這個 MVP 要做的事情就是：

- 把分散的每日行為，轉成可觀察的狀態
- 把感覺，變成可以討論的依據

2.4 MVP 的最小使用流程

這個系統刻意設計得非常簡單。

一個使用者只會做三件事：

1. 紀錄今天是否有發生某個行為
2. 查看一段時間內的節奏變化
3. 看到系統給出的狀態判斷

沒有設定地獄，沒有複雜操作。

因為 MVP 的目的是驗證價值，不是炫耀功能。

2.5 為什麼這個 MVP 一定需要後端

如果這個系統只存在前端，會立刻遇到問題：

- 資料一重整就消失
- 無法累積歷史紀錄
- 無法比較不同時間區段
- 無法做一致的狀態判斷

這些問題的本質不是技術限制，而是**責任放錯地方**。節奏判斷、狀態計算、資料保存，不應該由畫面層來負責。

2.6 前端在這個 MVP 中的角色

在這個 MVP 裡，前端的責任非常清楚：

- 接收使用者輸入
- 呈現結果與趨勢
- 不做任何關鍵判斷

前端的任務是讓人「看得懂現在的狀態」，而不是決定狀態本身。

2.7 後端在這個 MVP 中的角色

後端負責的是：

- 儲存時間序列資料
- 計算節奏指標
- 回傳狀態結果

這些工作有一個共同特徵：它們需要一致性、可回溯性、可調整性。

這也是為什麼，後端在 MVP 階段就已經是必要角色。

2.8 MVP 的「狀態」設計，比功能更重要

這個 MVP 不追求大量數字。它刻意把結果收斂成狀態，例如：

- 穩定
- 波動
- 斷裂風險

這樣的設計，讓系統可以：

- 支援不同使用情境
- 被人快速理解
- 成為決策參考，而不是資料堆疊

2.9 這個 MVP 如何延伸成不同產品

同一套核心結構，可以延伸成完全不同的產品樣貌：

- 個人學習節奏追蹤
- 創作者產出穩定度觀察
- 小團隊執行力監控

資料結構幾乎不變，改變的是「誰在看這個狀態」與「要做什麼判斷」。

2.10 為什麼這個 MVP 適合當 Week 3 主線

這個 MVP 有三個特點：

- 功能不多，但責任清楚
- 架構簡單，但真實世界會用到
- 可以自然銜接部署、BYOK、商業評估

它不是為了炫技，而是為了讓你理解：一個產品，從 Demo 走向真實使用時，結構會發生什麼變化。

2.11 這個 MVP 的最低完成標準

前端必須做到：

- 能讓使用者記錄每日行為
- 能顯示一段時間內的變化
- 不在前端做任何節奏或狀態判斷

後端必須做到：

- 能儲存每日行為紀錄
- 能依時間回傳資料
- 能回傳一個「狀態結果」，而不是一堆數字

刻意不需要做到：

- 使用者登入
- 權限管理
- 複雜設定
- 商業化流程

如果你現在的系統能清楚滿足以上條件，代表你已經完成從 Demo 邁向 MVP 的第一個關卡。

2.12 帶著這個 MVP，進入下一章

在下一章，我們會進一步拆解：

- 為什麼前端與後端不能混在一起
- 什麼責任放在前端會出事
- 什麼責任一定要交給後端

請先記住這個判斷基準：**問題如果跟「一致性、判斷、責任」有關，那它通常不屬於前端。**

Chapter 3 前端與後端：不是技術選擇，是責任切分

3.1 為什麼前端與後端一定要分清楚

在 Demo 階段，很多事情可以模糊處理：

- 邏輯寫在畫面裡
- 資料存在記憶體
- 重整就重來

但只要一個系統開始被視為 MVP，這些模糊地帶就會變成風險來源。前端與後端的分工，不是為了技術優雅，而是為了**責任清楚**。

3.2 一個簡單的判斷原則

在判斷某件事該放在哪裡之前，請先問這個問題：如果這件事做錯了，影響的是「畫面呈現」，還是「系統判斷結果」？

- 影響畫面 → 前端責任
- 影響判斷結果 → 後端責任

這個原則，會在後面所有 Web App 中反覆出現。

3.3 前端在 MVP 中該負責的事情

在 Week 3 的所有案例中，前端的角色都很一致。

前端應該負責：

- 收集使用者輸入
- 顯示系統回傳的結果
- 提供清楚的操作流程

前端的價值在於：讓人看得懂現在發生了什麼。

3.4 前端不該承擔的事情

一旦系統進入 MVP 階段，前端不該負責：

- 判斷使用者狀態
- 計算關鍵指標
- 決定通過或不通過
- 保存長期歷史資料

如果這些邏輯寫在前端，會發生三件事：

1. 每個人看到的結果可能不同
2. 重整或換裝置就失效
3. 商業與責任邏輯暴露在畫面層

3.5 後端在 MVP 中存在的真正理由

後端存在，不是因為技術比較難，而是因為有些事情必須集中處理。

後端負責的是：

- 資料儲存
- 一致性
- 判斷邏輯
- 可回溯性

這些事情的共同特徵是：一旦出錯，後果不只影響單一使用者。

3.6 用主線 MVP 看前後端分工

以今天的主線 MVP 為例：

- 前端負責：
 - 記錄每日行為
 - 顯示節奏變化
- 後端負責：
 - 儲存每日紀錄
 - 計算節奏
 - 回傳狀態結果

如果節奏判斷寫在前端，那麼每一個裝置，都會活在不同版本的世界裡。

3.7 為什麼「一致性」一定是後端責任

只要一個系統符合以下任一條件：

- 多人使用
- 跨裝置使用
- 需要歷史回溯

那麼「一致性」就不再是 UI 問題。

一致性意味著：

- 同一筆資料
- 同一套判斷
- 同一個結果

這些條件，前端無法獨立保證。

3.8 常見錯誤：把後端邏輯藏在前端

在實作中，最常見的錯誤不是「不會寫後端」，而是「先把邏輯塞進前端再說」。這樣做短期看起來很快，

但會導致：

- 無法擴充
- 無法除錯
- 無法說清責任

當你開始覺得「這段邏輯有點重要」，那它通常已經不適合放在前端。

3.9 在本課中，哪些 Web App 一定需要後端

接下來的 Demo 章節中，你會看到多種 Web App：

- 即時同步系統
- 決策型系統
- 長期記憶型系統

它們的共同點不是功能複雜，而是責任無法分散在畫面層。

3.10 這一章你應該能做到的判斷

完成本章後，你應該能清楚回答：

- 這個功能如果寫在前端，會不會出事？
- 這個判斷如果每個人都不一樣，後果是什麼？
- 這個責任，我現在有沒有能力承擔？

如果你能回答這些問題，代表你已經具備進入後端與 MVP 實作的心理準備。

3.11 帶著這個分工，進入下一章

在下一章，我們會進一步討論：

- 為什麼 BYOK 是產品設計問題
- API Key、成本與責任之間的關係
- 哪些設計在 Demo 可以，MVP 不行

請記住這一章的核心結論：**問題一旦牽涉到「判斷與責任」，就不該留在前端。**

Chapter 4 BYOK 是產品設計問題

4.1 什麼是 BYOK (Bring Your Own Key)

BYOK 的意思是：使用者在使用你的系統時，必須自己提供 API Key (例如 AI、第三方服務的金鑰)，而不是由你幫所有人代付或代管。

這件事乍看之下像是技術選擇，但實際上，它會直接影響：

- 成本怎麼算
- 風險誰扛
- 這個產品能不能活下來

4.2 為什麼在 Demo 階段很容易忽略 BYOK

在 Demo 階段，常見的做法是：

- 先用自己的 API Key 撐著跑
- 反正只有自己用
- 反正流量很小

這在 Demo 階段是合理的，但只要你開始進入 MVP，問題就會出現。

因為你已經不是在證明「跑得動」，而是在讓系統「被使用」。

4.3 如果不想 BYOK，會發生什麼事

假設你不設計 BYOK，而是選擇：

- 所有使用者共用一把 Key
- 或金鑰直接寫在系統裡

那你會立刻面臨這些問題：

- 用量一高，成本直接爆炸
- 出問題時，你不知道是誰用的
- 金鑰外流，你要負全責

這些風險，不會因為你只是做 MVP 就消失。

4.4 BYOK 本質上是在做一個責任切分

BYOK 真正在解決的不是技術問題，而是這一題：

這個系統所產生的成本，到底是「使用者的責任」，還是「產品擁有者的責任」？

一旦你讓使用者帶自己的 Key，你就清楚地把責任切開了。

4.5 常見的三種 BYOK 設計方式

在實務中，BYOK 大致會出現三種設計方式。

4.5.1 完全前端 BYOK

API Key 直接由使用者在前端輸入，前端直接呼叫第三方服務。

優點：

- 結構最簡單
- 成本與責任完全在使用者

限制：

- 金鑰暴露風險高
- 不適合多人或商業產品

這種方式只適合：

- 個人工具
- 練功型作品
- 非正式使用情境

4.5.2 後端轉接型 BYOK (最適合 MVP)

使用者提供 Key，但由後端代為呼叫第三方服務。

優點：

- 金鑰不暴露在前端
- 可以加上限制與保護
- 結構仍然可控

代價：

- 你需要負責最小的系統穩定性
- 必須思考錯誤處理與限制

這是課程 MVP 階段最推薦的做法。

4.5.3 平台代管型 (不適合課堂 MVP)

所有使用者共用平台的 Key，由產品方統一吸收成本。

這種方式會立刻帶來：

- 成本不可控
- 濫用風險
- 商業責任壓力

這不是 MVP 階段該碰的設計。

4.6 為什麼 BYOK 一定要搭配後端設計

只要你選擇「後端轉接型 BYOK」，後端就必須負責：

- 接收使用者 Key
- 安全地使用它
- 控制呼叫頻率
- 回傳結果或錯誤

這些責任，前端不適合也無法承擔。這也是為什麼 BYOK 與後端設計是綁在一起的議題。

4.7 在本課的 Web App 中，BYOK 會怎麼出現

在接下來的實作章節中，你會看到：

- AI 輔助型功能
- 決策型 Web App
- 分析與建議系統

這些功能都可能涉及付費 API。

本課的原則是：

- Demo 可以先不用
- MVP 一定要想
- 只要會產生成本，就不能裝沒看到

4.8 BYOK 沒想清楚，商業評估就一定失真

如果你在評估一個產品時：

- 不知道每個使用者會花多少錢
- 不知道成本落在誰身上
- 不知道流量放大後會怎樣

那任何商業判斷，都是建立在假設之上，BYOK 是商業評估的前置條件。

4.9 本章你應該能做出的判斷

完成本章後，你應該能清楚回答：

- 這個 Web App 有沒有產生成本？
- 這個成本該不該由使用者承擔？
- 如果不 BYOK，後果是什麼？
- 這個系統我扛不扛得住？

如果你答得出來，代表你已經準備好進入實作章節。

4.10 帶著 BYOK 的設計意識，進入實作

接下來的 Chapter 5，會正式開始 Web App 的實作與 Demo。

在每一個案例中，BYOK 都會是一個你必須主動思考、而不是事後補救的設計決策。

Chapter 5 Web App 實作：用 Q 做出可部署的 MVP

5.1 本章你在做的不是練習，是一個 MVP

從這一章開始，你不再只是寫「會動的東西」，你要做的是一個可以被別人連線使用的 Web App。

這代表三件事必須同時成立：

1. 前端與後端真的接在一起
2. 系統可以部署到雲端
3. 你說得出來每一層在負責什麼

本章不提供完整程式碼，你會透過 Developer Q 指令，一步一步把系統「生出來」。

5.2 本章實作方法（請照順序）

每一個 Web App，都嚴格照這個流程：

1. 用 Q 建立專案骨架
2. 用 Q 定義 API 契約
3. 用 Q 產出後端
4. 用 Q 產出前端
5. 用 Q 協助前後端合體
6. 用 Q 完成雲端部署

只要順序亂掉，卡關機率會大幅提高。

5.3 本章你可以選擇的部署方式（先講清楚）

在本課中，你可以選擇以下任一組合：

選項 A：Cloud Run（後端） + 任意靜態前端

最接近真實產品，最推薦

選項 B：Firebase Hosting（前端） + Cloud Functions（後端）

Firebase 生態系新手友善

選項 C：Firebase Hosting（前端） + Cloud Run（後端）

實務常見混合型架構

注意：**Firebase Hosting** 只能放前端，不能直接放後端 API。

Web App #1 | 主線 MVP：節奏型行為追蹤系統（必做）

5.4 這個 Web App 的最低完成標準

你只需要做到以下四件事：

- 前端可以送出每日紀錄
- 後端可以儲存資料
- 後端會回傳一個 status
- 前端只顯示，不做判斷

做到這裡，就已經是 MVP。

5.5 Step 1：用 Q 建立專案骨架

把下面這段貼給 Developer Q：

你是全端協作 AI。我要做一個「節奏型行為追蹤」Web App，包含前端與後端。

請先不要寫完整程式碼，只幫我設計「專案骨架與檔案結構」，包含：

- frontend/ (純靜態前端，可用 Live Server)
- backend/ (提供 HTTP API)
- README.md (說明本地如何啟動)

請列出：

- 每個資料夾與檔案名稱
- 每個檔案的用途說明

等我確認後，再逐檔案產出內容。

你的任務：建立資料夾與空檔案，不寫內容。

5.6 Step 2：用 Q 定義 API 契約 (前後端合體核心)

請幫我為「節奏型行為追蹤」設計最小 API 契約。

需求：

- 1) 新增每日紀錄 (POST)
- 2) 取得最近 N 天資料與狀態 (GET)

每個 API 請清楚列出：

- method
- path
- request JSON 範例
- response JSON 範例
- status 的意義 (由後端計算)

請用「文件格式」輸出，不要寫實作程式碼。

檢核點：你現在應該完全知道前端要送什麼、後端要回什麼。

5.7 Step 3 : 用 Q 產出後端 (先能跑)

根據剛剛的 API 契約，請你開始協助我產出 backend/ 的程式碼。

規則：

- 一次只輸出一個檔案 (檔名 + 完整內容)
- 從後端入口檔開始
- 資料可以先存在記憶體或簡單檔案
- status 的判斷邏輯一定要在後端
- 請同時說明如何啟動後端

請在每個檔案後附上：「這個檔案在系統中負責什麼」

檢核點：

- 後端可以啟動
- API 打得到 (用瀏覽器或工具測)

5.8 Step 4：用 Q 產出前端（只做輸入與顯示）

現在請根據同一份 API 契約，協助我產出 frontend/ 的程式碼。

需求：

- 一次輸出一個檔案
- 提供輸入表單 (date + did)
- 呼叫後端 API
- 顯示 series 與 status
- 前端不得自行計算 status
- 請預留 API_BASE_URL 設定

請從 HTML 檔開始。

檢核點：

- 前端畫面能打開
- 點擊按鈕會發送請求（即使先失敗也沒關係）

5.9 Step 5：用 Q 完成前後端合體與測試

我已經有前端與後端程式碼，請協助我完成「合體」。

請你用步驟清單的方式說明：

- 1) 前端 `API_BASE_URL` 應如何設定
- 2) 本地前後端如何同時跑
- 3) 如果遇到 `CORS`，要修改哪裡（只說修改點）
- 4) 最小測試流程（`POST` → `GET` → 前端顯示）

請不要重貼整包程式碼。

5.10 Step 6 : 用 Q 部署後端 (Cloud Run 或 Firebase)

選 Cloud Run (推薦)

你是雲端部署教練。我要把 backend/ 部署到 Cloud Run 。

請用新手可跟著做的步驟清單說明：

- 1) 需要準備的 GCP 環境
- 2) 是否需要 Dockerfile (需要的話說明結構)
- 3) 部署指令與設定重點
- 4) 如何取得部署後的公開 URL
- 5) 如何測試 API 是否正常

或選 Firebase Functions

你是 Firebase Functions 教練。我要把 backend API 部署成 HTTP Functions 。

請一步一步帶我完成：

- 1) 初始化 functions
- 2) 將 API 掛成 HTTP function
- 3) 本地測試
- 4) 部署
- 5) 取得公開 URL 與 CORS 設定

5.11 Step 7：部署前端並完成對外展示

你是前端部署教練。我要把 frontend/ 部署成公開網站。

請教我：

- 使用 Firebase Hosting 的完整流程
- API_BASE_URL 要如何改成雲端後端網址
- 如何驗證前端真的打到雲端後端

5.12 主線 MVP 完成標準（請自我檢核）

你已完成本 Web App，如果你可以說出：

- 後端部署在哪
- API 的公開 URL 是什麼
- 前端如何連到後端
- status 為什麼不能在前端算

Web App #2、#3（延伸 Demo）

接下來的兩個 Web App（匿名回饋牆、決策型 Web App），完全沿用同一套

Q 驅動流程：

- 換需求
- 換 API 契約
- 換部署選擇

你不需要學新技術，只是在練「產品型實作節奏」。

5.13 本章你真正學會的是什麼

你學到的不是某個框架，而是：

- 如何把需求變成 Q 指令
- 如何讓 AI 幫你寫對位置的程式
- 如何完成一個可部署的 MVP

Web App #2 | 即時匿名回饋牆 (多人同步型)

核心能力：世界同步、資料一致性、後端存在必要性

#2.1 這個 Web App 在做什麼

這是一個讓多個人同時看到同一份內容的 Web App。

使用者可以：

- 匿名送出一段文字
- 看到所有人送出的內容列表
- 重新整理後資料仍存在

這個產品的重點不是即時 UI，而是：「所有人看到的是同一個世界」

#2.2 最低完成標準 (做到這裡就算過關)

- 任一使用者送出訊息
- 其他人重新整理頁面仍能看到
- 訊息順序一致
- 前端不負責排序

#2.3 Step A | 用 Q 建立專案骨架

你是全端協作 AI。我要做一個「即時匿名回饋牆」Web App (前端 + 後端)。

請先不要寫程式碼，只幫我設計專案骨架與檔案結構，包含：

- frontend/
- backend/
- README.md

請列出：

- 每個資料夾與檔案名稱
- 每個檔案的用途

限制：

- 前端為純靜態 (HTML/CSS/JS)
- 後端提供 HTTP API

等我確認後，再逐檔案產出內容。

#2.4 Step B | 用 Q 定 API 契約

請幫我為「即時匿名回饋牆」設計最小 API 契約。

需求：

- 1) 新增訊息 (POST)
- 2) 取得訊息列表 (GET)

每個 API 請列出：

- method
- path
- request JSON 範例
- response JSON 範例
- 排序規則 (請說明由誰負責)

限制：

- 不需要登入
- 訊息順序必須一致

#2.5 Step C | 用 Q 產出後端 (資料一致性)

根據剛剛的 API 契約，請協助我逐步產出 backend/ 的程式碼。

規則：

- 一次輸出一個檔案 (檔名 + 內容)
- 後端必須負責：
 - 儲存訊息
 - 依時間排序
- 資料可先用記憶體或簡單檔案
- 請說明如何啟動後端
- 請在每個檔案後說明「它的責任」

請從後端入口檔開始。

檢核點：

- 後端啟動成功
- GET 可以拿到完整列表

#2.6 Step D | 用 Q 產出前端 (只顯示)

請根據同一份 API 契約，協助我產出 frontend/ 的程式碼。

需求：

- 一次輸出一個檔案
- 前端可以：
 - 輸入一段文字
 - 呼叫 POST API
 - 呼叫 GET API 顯示列表
- 前端不得排序資料
- 請預留 API_BASE_URL 設定

請從 HTML 檔開始。

#2.7 Step E | 用 Q 合體與測試

我已經有前端與後端，請協助我完成合體與驗證。

請用步驟清單說明：

- 1) API_BASE_URL 如何設定
- 2) 本地前後端如何同時跑
- 3) 最小測試流程 (送一筆 → 取得列表 → 重整)
- 4) 常見錯誤排查 (404、CORS、順序錯亂)

請不要貼完整程式碼。

#2.8 Step F | 部署 (選一種)

Cloud Run (後端) + Firebase Hosting (前端)

我要把「匿名回饋牆」部署成可對外使用的版本。
後端用 Cloud Run，前端用 Firebase Hosting。

請一步一步告訴我：

- 1) 後端部署到 Cloud Run 的流程
- 2) 如何取得後端 API URL
- 3) 前端如何改 API_BASE_URL
- 4) Firebase Hosting 的部署流程
- 5) 如何驗證部署成功

Web App #3 | 決策型 Web App (後端判斷)

核心能力：規則集中、商業邏輯不可前端化

#3.1 這個 Web App 在做什麼

這是一個「前端送資料、後端回傳判斷結果」的系統。例如：

- 是否通過審核
- 是否建議執行
- 是否達到門檻

前端不知道規則，只顯示結果與理由。

#3.2 最低完成標準

- 前端可送出條件
- 後端回傳 decision + reason
- 規則只存在後端
- 規則可調整，不影響前端

#3.3 Step A | 用 Q 建立專案骨架

你是全端協作 AI。我要做一個「決策型 Web App」(前端 + 後端)。

請先幫我設計專案骨架與檔案結構：

- frontend/
- backend/
- README.md

限制：

- 前端不包含任何判斷規則
- 後端負責所有 decision logic

#3.4 Step B | 用 Q 定 API 契約

請為「決策型 Web App」設計 API 契約。

需求：

- 提交條件 (POST)
- 回傳 decision 與 reason

請列出：

- request JSON 範例
- response JSON 範例
- decision 的可能值
- reason 的用途說明

#3.5 Step C | 用 Q 產出後端 (規則集中)

根據 API 契約，請逐步協助我產出 backend/ 的程式碼。

規則：

- 一次輸出一個檔案
- 規則邏輯只存在後端
- 規則請集中管理 (可用簡單 if/else)
- 請說明如何調整規則而不動前端

#3.6 Step D | 用 Q 產出前端 (只顯示結果)

請根據 API 契約，產出 frontend/ 的程式碼。

需求：

- 表單讓使用者輸入條件
- 呼叫後端 API
- 顯示 decision 與 reason
- 前端不得推測規則
- 預留 API_BASE_URL 設定

#3.7 Step E | 合體與驗證

我已完成前端與後端，請協助我完成合體。

請列出：

- 1) API_BASE_URL 設定方式
- 2) 本地測試流程 (送條件 → 看結果)
- 3) 常見錯誤 (decision 為空、格式錯誤)

#3.8 Step F | 部署 (推薦 Cloud Run)

我要把「決策型 Web App」後端部署到 Cloud Run，前端部署為靜態網站。

請一步一步帶我完成：

- 後端部署
- 取得公開 API URL
- 前端改連線設定
- 驗證 decision 正確回傳

這三個 Web App，看起來完全不同，但你用的是同一套 Q 驅動流程。

Chapter 6 Git 同步與交付：讓你的 Web App 不是只活在你電腦

6.1 為什麼「能跑」不等於「完成」

在 Chapter 5，你已經做到一件很重要的事：前端 + 後端 + 部署，全都能動。但這還不代表你完成了一個產品。

如果出現下面任何一種情況，它就還沒完成：

- 換一台電腦就跑不起來
- 別人不知道怎麼啟動
- 你不在場就沒人敢動
- 改壞了也回不去

這些問題，不是技術不夠，而是**沒有交付結構**。Git 在這一章的角色，就是解決這件事。

6.2 本章你要做到的三件事

完成 Chapter 6，你必須能做到：

1. 把專案同步到 Git
2. 讓別人照 README 跑起來
3. 發生錯誤時，有回頭路

這一章不是要你變 Git 專家，而是讓你**不再是單點故障**。

6.3 Step 1 | 用 Q 幫你建立「可交付」的 Git 專案結構

在你的專案根目錄 (例如 vibe-coding-week3/)，把下面這段貼給

Developer Q：

你是 Git 與交付顧問。

我現在有一個全端 Web App 專案 (frontend/ + backend/)。

請協助我檢查並優化專案結構，使它適合被同步到 Git 與交付給他人。

請輸出：

- 1) 建議的專案根目錄結構 (包含 frontend、backend、README)
- 2) 哪些檔案「一定不能 commit」
- 3) 哪些檔案「一定要有」
- 4) 常見新手在交付時會漏掉的項目清單

請用檢查清單形式輸出，不要貼程式碼。

你的任務：對照清單，確認你的資料夾結構沒有亂長。

6.4 Step 2 | 用 Q 幫你寫 .gitignore (避免災難)

請根據一個「純前端 + Node 後端」的 Web App，
幫我產出一份合適的 .gitignore 建議清單。

需求：

- 不要 commit API key、環境變數
- 不要 commit build 或暫存檔
- 請逐條解釋「為什麼要忽略」

請用「註解說明版」輸出。

你要做的事：

1. 建立 .gitignore
2. 確認沒有把金鑰或雲端設定推上去

6.5 Step 3 | 用 Q 幫你寫「真的有人看得懂」的 README

README 是交付的核心文件，沒有 README，就等於沒交付，把下面貼給 Developer Q：

你是技術文件編輯。請幫我為一個全端 Web App 專案撰寫 README.md 草稿。

請包含以下段落：

- 1) 這個專案在做什麼（一段話）
- 2) 系統架構（前端 / 後端 / 部署位置）
- 3) 本地啟動方式（一步一步）
- 4) 環境需求（Node 版本、工具）
- 5) API Base URL 如何設定
- 6) 常見錯誤與排查方式
- 7) 目前的限制（刻意不做的事）

限制：

- 對象是假設「沒參與開發的人」
- 不要假設讀者會通靈

檢核點：你能把 README 丟給別人，他照著做真的跑得起來。

6.6 Step 4 | 用 Q 建立「安全的环境變數策略」

請協助我設計一個「安全的环境變數使用策略」，適用於：

- 本地開發
- Git 同步
- 雲端部署 (Cloud Run / Firebase)

請說明：

- 1) 哪些設定該放在環境變數
- 2) 哪些可以寫在前端 config
- 3) 本地與雲端如何區分
- 4) 常見錯誤 (例如把 key 寫死)

請用概念 + 步驟說明，不要直接貼實作碼。

這一步的目的只有一個：讓你之後敢公開 Repo。

6.7 Step 5 | 用 Q 規劃「不會後悔的 commit 節奏」

你是版本控制顧問。請幫我規劃一個適合新手的 Git commit 節奏與原則。

請說明：

- 1) 什麼時候該 commit
- 2) 一次 commit 裡面應該包含什麼
- 3) 常見的壞 commit 習慣
- 4) 推薦的 commit message 寫法 (範例)

限制：

- 以「可回溯、可教學」為目標

你要做到的不是 commit 很多，而是 每一次 commit 都有意義。

6.8 Step 6 | 用 Q 模擬「交付給別人」的檢查流程

請你扮演「接手這個專案的同事」。

根據我的專案結構與 README，請幫我檢查：

- 1) 有沒有看不懂的地方
- 2) 哪些步驟可能會卡關
- 3) 哪些資訊不夠清楚
- 4) 如果我要讓新手接手，最該補哪一段

請用審查清單方式回饋。

你會第一次知道：你寫的東西是不是只有你自己懂。

6.9 本章完成標準（請誠實對自己）

你完成 Chapter 6，如果你能做到：

- 專案已同步到 Git
- 沒有把敏感資訊推上去
- README 真的能讓別人跑起來
- 出問題時你知道回到哪個 commit

做到這裡，你的 Web App 才真的「完成」。

6.10 為什麼這一章比寫程式還重要

因為多數人卡住的不是「不會寫」，而是：

- 寫完不敢給人看
- 壞了不知道怎麼救
- 做了卻不能交付

你現在學的是：**怎麼讓一個東西，離開你之後還活得下去。**

Chapter 7 部署與展示：把你的 Web App 變成「能給人看的東西」

7.1 為什麼「能部署」不等於「能展示」

在 Chapter 5 與 6，你已經做到：

- 前端與後端能跑
- 有雲端 URL
- 有 Git 與 README

但如果你現在把網址丟給別人，對方很可能會：

- 不知道要怎麼用
- 不敢亂按
- 用完不知道重點是什麼
- 看不出這是不是一個產品

展示 (Demo) 不是部署的副產品，而是一個刻意設計的行為。

7.2 本章你要完成的三個展示任務

完成 Chapter 7，你的 Web App 必須同時滿足：

1. 第一次打開就知道在幹嘛
2. 不用你在旁邊解說也能操作
3. 看得出這個產品的核心價值

7.3 展示前你一定要先做的「收斂」

在展示之前，請先確認這三件事：

- 你現在只展示「一條主線流程」
- 你暫時不展示所有功能
- 你知道「觀眾看完要記住什麼」

如果你自己都說不出來，那展示只會變成災難。

7.4 Step 1 | 用 Q 幫你定「展示版本的目標」

把下面這段貼給 Developer Q：

你是產品展示顧問。我有一個已部署的 Web App，請協助我定義「展示版 Demo 的目標」。

請幫我輸出：

- 1) 這個展示版要讓觀眾理解的一句話
- 2) 展示時唯一的一條操作流程 (step by step)
- 3) 這次展示刻意不展示的功能清單
- 4) 觀眾看完後，應該留下的三個印象

限制：

- 假設觀眾不懂技術
- 假設你只有 2-3 分鐘

7.5 Step 2 | 用 Q 幫你調整「展示專用 UI」

展示用 UI 跟開發用 UI 不一樣。貼給 Developer Q：

你是 UI/UX 檢視顧問。這是一個要展示給外人看的 Web App。

請幫我檢視目前的畫面，並提出「最小修改建議」，讓它更適合 Demo。

請聚焦在：

- 初次打開是否看得懂？
- 按鈕與文字是否有引導性？
- 是否有任何會讓人困惑的元素？

限制：

- 不重寫整個 UI
- 只給「必要調整建議」

你的任務：只做最小修改，不追求美。

7.6 Step 3 | 用 Q 幫你準備「展示說詞」

很多 Demo 失敗，不是產品不行，而是說詞亂掉。貼給 Developer Q：

你是 Demo 簡報教練。請幫我為這個 Web App 準備一段「90 秒展示說詞」。

請包含：

- 1) 開場一句話（在幹嘛）
- 2) 一個真實使用情境
- 3) 現場操作的三個重點步驟
- 4) 結尾一句「這個東西的價值」

限制：

- 對象是非工程背景
- 用白話，不用技術名詞

7.7 Step 4 | 用 Q 設計「展示用測試資料」

你是 Demo 測試資料設計師。請幫我設計一組「展示專用測試資料」。

需求：

- 可以快速帶出產品重點
- 不需要太多資料
- 每一步操作都有明確效果

請輸出：

- 測試資料內容
- 建議操作順序
- 每一步應該讓觀眾看到什麼

7.8 Step 5 | 用 Q 做「展示前最後檢查」

你是 Demo QA 檢查員。請幫我做一份「展示前檢查清單」。

請包含：

- 網址是否正確
- API 是否正常
- 是否有殘留測試用錯誤訊息
- 是否有會讓 Demo 翻車的地方

請用勾選清單格式。

7.9 不同 Web App 的展示重點提醒

主線 MVP (節奏追蹤)

- 重點：狀態變化，不是資料量
- 不要解釋演算法

匿名回饋牆

- 重點：多人看到同一份資料
- 不要強調即時技術細節

決策型 Web App

- 重點：前端不知道規則
- 不要公開判斷邏輯

7.10 本章完成標準

你完成 Chapter 7，如果你能做到：

- 把網址丟給朋友，他不用問你就能操作
- 你能在 90 秒內 Demo 完一條流程
- 對方能說出「這個產品在幹嘛」

如果這三件事成立，代表你的 Web App 已經「可展示」。

7.11 為什麼展示是產品能力的一部分

因為產品不是寫給自己用的，如果你無法展示清楚，那就代表：

- 產品定位不清楚
- 使用流程不清楚
- 價值不清楚

展示，是逼你把這三件事講清楚。

Chapter 8 Vibe Coding 常見翻車點與修正方式

8.1 為什麼「會用 Q」反而更容易翻車

當你開始大量使用 Developer Q 之後，很容易出現一種錯覺：「反正 Q 會幫我補齊。」

這個錯覺，會讓很多專案死在一個很尷尬的位置：

- 程式碼越來越多
- 但你越來越說不清楚在幹嘛
- 出問題時不知道該問哪一段

Vibe Coding 的風險，不在於寫錯，而在於失去結構感。

8.2 翻車類型一：指令太大，一次要太多

常見症狀

- 一次丟給 Q 一整段需求
- Q 回了一大包
- 你不知道該貼哪、改哪、刪哪

為什麼會翻

因為你跳過了「拆任務」這一步，直接要求 AI 給你結果。

修正方式 | 把指令拆成「只做一件事」

請你只做一件事： _ _ _ _ _

限制：

- 不要延伸到其他功能
- 不要幫我優化或加新需求

判斷標準很簡單：如果 Q 回的東西，你不知道該放哪個檔案，那這個指令就太大了。

8.3 翻車類型二：沒有 API 契約就開始寫

常見症狀

- 前端先寫
- 後端補不回來
- 最後兩邊都在猜對方在幹嘛

為什麼會翻

因為你讓 AI 同時當「設計師」與「工程師」，但你自己沒有站在任何一邊。

修正方式 | 先叫 Q 只寫「契約文件」

請你現在不要寫任何程式碼，只幫我產出 API 契約文件。

內容包含：

- endpoints
- request / response JSON
- 欄位意義說明

只要契約沒定完，任何程式碼都是暫時的。

8.4 翻車類型三：前端開始偷做後端的事

常見症狀

- status 在前端算
- decision 在前端判
- 規則散落在 UI 裡

為什麼會翻

因為「這樣比較快看到結果」，但你其實是在犧牲產品結構。

修正方式 | 強制 Q 檢查責任歸屬

請你檢查目前這段功能設計，

逐條標示：

- 哪些是前端責任
- 哪些是後端責任

如果有責任放錯地方，請直接指出並說明原因。

如果 Q 指出「這段不該在前端」，你要信它，不要硬拗。

8.5 翻車類型四：改一點東西，整個系統崩

常見症狀

- 改一個欄位
- API 全掛
- 前端報錯看不懂

為什麼會翻

因為你沒有「修改邊界」。

修正方式 | 用 Q 做 Atomic Edit

我要修改一個非常小的地方： _ _ _ _ _

限制：

- 只允許修改指定檔案
- 只修改與此需求直接相關的程式
- 不要順手重構其他地方

修改完成後，請列出你改了哪幾行，為什麼。

這個指令，是避免 AI「順便幫你搞更大包」的關鍵。

8.6 翻車類型五：錯了，但不知道錯在哪一層

常見症狀

- 404
- CORS
- 資料空的
- 但你不知道是前端、後端、部署哪裡出問題

修正方式 | 強制分層 Debug

我現在有一個錯誤現象： _ _ _ _ _

請你不要直接給我解法，

而是依序幫我檢查：

- 1) 前端層可能的問題
- 2) API / 後端層可能的問題
- 3) 部署 / 環境層可能的問題

每一層請給我：

- 檢查方式
- 預期看到的結果

這會讓你重新拿回「系統視角」。

8.7 翻車類型六：Q 寫得越來越不像你

常見症狀

- 檔案風格混亂
- 命名不一致
- 每次生成像不同人寫的

為什麼會翻

因為你沒有「約束風格」。

修正方式 | 把風格寫進指令

在接下來所有產出中，請遵守以下風格：

- 命名規則： _ _ _ _
- 檔案結構： _ _ _ _
- 註解語氣： _ _ _ _

如果你覺得不適合，請先詢問我，不要自行更改。

AI 很會配合，但前提是你有說清楚。

8.8 本章的核心不是避免錯誤，而是避免失控

錯誤不可怕，失控才可怕。

Vibe Coding 真正要學的不是「讓 AI 幫你寫」，而是：你還能不能清楚知道現在在做什麼。

8.9 本章完成標準（請自我檢核）

你完成 Chapter 8，如果你能做到：

- 每次卡關都能說出「卡在哪一層」
- 每次改動都能限制範圍
- Q 回的內容你知道要放哪

那你就已經不只是「會用 AI 寫程式」，而是在駕馭 AI 寫程式。

Chapter 9 商業評估：AI Coding 產品要怎麼活下來

9.1 為什麼 AI Coding 產品「做得出來」通常不是問題

做到這一章，你已經證明三件事：

- 你能用 AI 把產品做出來
- 你能完成前後端、部署、展示
- 你不是只會寫 Demo

那接下來真正的問題會變成：「這個東西，怎麼在現實世界中存在？」

AI Coding 產品最大的誤判，不是技術力，而是商業結構想錯層級。

9.2 AI Coding 產品最常見的三個錯誤期待

在談商業模式之前，先把三個幻想打掉。

錯誤一：功能做出來，自然會有人付費

事實是：沒有人會為「功能」付費，只會為「減少風險或時間」付費。

錯誤二：我可以先免費，之後再想

AI 產品一旦牽涉到 API 成本，「免費」其實等於「你在補貼陌生人」。

錯誤三：這個很厲害，一定有人買

AI Coding 很容易「對工程師很厲害」，但對使用者只是「看不懂」。

9.3 AI Coding 產品的五種常見商業模式（你一定要認得）

以下不是理論模型，而是你現在這種產品最可能走的路。

9.3.1 工具型（Tool SaaS）

長相

- 一個明確功能
- 解決一個小但重複的問題

適合的產品

- 決策型 Web App
- 自動判斷、篩選、整理工具

付費邏輯

- 月費 / 年費
- 依使用量限制

風險點

- 使用頻率低
- 用完就走

判斷關鍵：「這個工具，使用者一個月會開幾次？」

9.3.2 BYOK 平台型（AI 功能層）

長相

- 核心價值不是模型
- 而是「流程、介面、判斷邏輯」

適合的產品

- AI 輔助決策
- AI 分析工具
- AI 教練型產品

付費邏輯

- 使用者自己出 API Key
- 你收平台使用費

優勢

- 成本風險低
- 可快速驗證

這是課堂 MVP 最推薦的商業路線。

9.3.3 專案型 / 顧問型 (Solution)

長相

- 不是賣產品
- 是賣「這套系統幫你解決什麼」

適合的產品

- 節奏追蹤
- 內部決策工具
- 部門系統

付費邏輯

- 一次性專案費
- 客製化

現實優點

- 最快變現
- 不需要規模化

很多 AI Coding MVP，最適合先走這條路。

9.3.4 內容 × 工具混合型

長相

- 工具本身不是賣點
- 「怎麼用、怎麼想」才是

適合的產品

- 教學型 AI 工具
- 創作者輔助系統

付費邏輯

- 課程
- 會員
- 模板 + 工具包

如果你本身是創作者或講師，這條路很合理。

9.3.5 展示型（不賣產品）

這不是失敗，而是一個**清楚的選擇**。

目標

- 展示能力
- 拿合作
- 換工作、案子、曝光

很多 AI Coding 作品，**最健康的結局就是停在這裡**。

9.4 用三個問題，快速篩選商業可能性

請對你的 Web App 回答下面三題：

1. 如果今天有 10 個人用，你會緊張嗎？
2. 如果有 100 個人用，你會破產嗎？
3. 如果有 1 個人願意付錢，你敢收嗎？

如果三題都是「不確定」，那這個產品暫時不適合商業化。

9.5 AI 成本結構，你一定要算的不是營收

AI 產品最常見的致命點是：用量一上來，成本先爆。

你至少要知道：

- 每一次 AI 呼叫，大概多少錢
- 一個使用者，一天會用幾次
- 你是不是在補貼「亂用的人」

如果你現在不想面對這件事，那 BYOK 就不是選項，而是必要。

9.6 用 Q 幫你選「最適合現在的商業定位」

你是 AI 產品策略顧問。我有一個 AI Coding Web App，請你協助我判斷「最適合現在階段的商業模式」。

請依序分析：

- 1) 這個產品目前最像哪一種類型 (工具 / BYOK / 專案 / 展示)
- 2) 為什麼不適合其他類型
- 3) 如果要商業化，最小可行的第一步是什麼
- 4) 哪些風險是我現在不該碰的

請用「保守建議」而不是理想化版本回答。

這個 Q 的目的不是讓你興奮，而是讓你**不要亂衝**。

9.7 本課真正想讓你帶走的商業觀念

如果你只記得一句話，請記住這個：

AI Coding 產品，不是比誰做得多，而是比誰「知道什麼時候不要做」。

9.8 結課總結

你現在具備的不是「寫 AI 產品的能力」，而是三個更稀缺的能力：

1. 能把 AI 變成系統
2. 能判斷責任與風險
3. 能選擇「停、走、轉彎」

這三件事，比任何一個爆紅 Demo 都值錢。

三種 AI 產品商業路線圖 (實作後的真實選項)

前提先講清楚：不是每一個 AI Web App 都該變成 SaaS。能活下來的關鍵，是你選對「現在這個階段」的商業路線。

路線一 | BYOK 工具型 (最適合課堂 MVP)

產品長相

- AI 是功能的一部分，不是賣點本身
- 使用者 自備 API Key (BYOK)
- 你提供的是：
 - 流程
 - 判斷邏輯
 - 使用體驗

對應的 Week 3 Web App

- 決策型 Web App
- AI 輔助判斷工具
- AI 教練型系統 (輕量)

商業結構

- 成本：
 - 幾乎為 0 (使用者自行負擔 AI 成本)
- 收費方式：
 - 月費
 - 一次性使用費
 - 進階功能解鎖
- 技術門檻：
 - 不需要大規模 infra

- MVP 就可驗證

什麼情況下適合選這條路

- 你現在不想扛 AI 成本
- 你想先驗證「有沒有人願意為這個流程付錢」
- 你不想被用量暴增拖垮

這條路的風險

- 使用者需要有技術理解 (API Key)
- 大眾市場不友善
- UI/UX 必須非常清楚

賣的是『判斷力與流程』，不是 AI token 。

路線二 | 專案型 / 顧問型 (最快變現)

產品長相

- Web App 是「證據」, 不是最終商品
- 真正賣的是：
 - 系統設計
 - 解法
 - 導入能力

對應的 Week 3 Web App

- 節奏型行為追蹤系統
- 內部決策工具
- 團隊用 AI 系統

商業結構

- 成本：
 - 可控
 - 可轉嫁給客戶
- 收費方式：
 - 一次性專案費
 - 顧問費
 - 客 a 導入費
- 技術門檻：
 - 不需要 scale
 - 需要說清楚價值

什麼情況下適合選這條路

- 你是接案者、顧問、內訓講師

- 你想要「先有現金流」
- 你不想 24 小時維運產品

這條路的風險

- 不可規模化
- 你本人容易成為單點故障
- 需要溝通與交付能力

這個系統不是賣給所有人，是賣給『現在這個客戶』

路線三 | 內容 × 工具混合型 (個人品牌最適合)

產品長相

- 工具本身不是主角
- 主角是：
 - 使用方法
 - 思考框架
 - 教學內容

對應的 Week 3 Web App

- 節奏追蹤系統 (教學版)
- AI 練功工具
- 示範型 Web App

商業結構

- 成本：
 - 低
 - 可控
- 收費方式：
 - 課程
 - 會員
 - 模板 + 工具包
- 技術門檻：
 - 工具只要「夠用」
 - 重點在教法與包裝

什麼情況下適合選這條路

- 你本身是創作者、講師、KOL

- 你想建立長期影響力
- 你不想維運複雜產品

這條路的風險

- 工具本身不值錢
- 內容品質壓力大
- 需要持續輸出

工具是入口，內容才是收入。

三條路線快速對照表

路線	最適合誰	AI 成本	是否可規模化	MVP 適配度
BYOK 工具型	想做產品的人	低	中	★★★★★
專案 / 顧問型	接案者、顧問	客戶負擔	低	★★★★★
內容 × 工具	創作者、講師	低	中	★★★★★

Week 3 做的不是三個 App，而是讓你知道：
你現在做出來的這個東西，最適合走哪一條路。